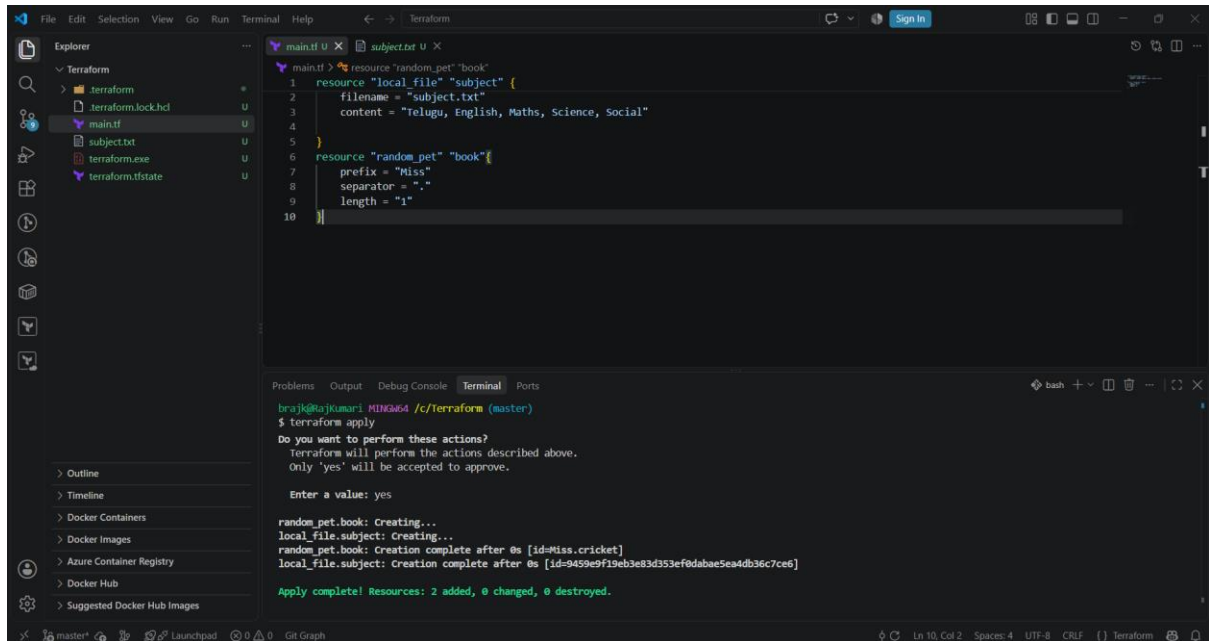


TERRAFORM TASK - 03

1. Execute the script shown in the video.



This screenshot shows the VS Code interface with a Terraform configuration file open. The Explorer sidebar on the left shows the project structure: Terraform, terraform.lock.hcl, main.tf, subject.txt, terraform.exe, and terraform.tfstate. The main editor displays the content of main.tf, which defines a local_file resource named 'subject' and a random_pet resource named 'book'. The terminal at the bottom shows the execution of the terraform apply command, which prompts for confirmation and then successfully creates the resources.

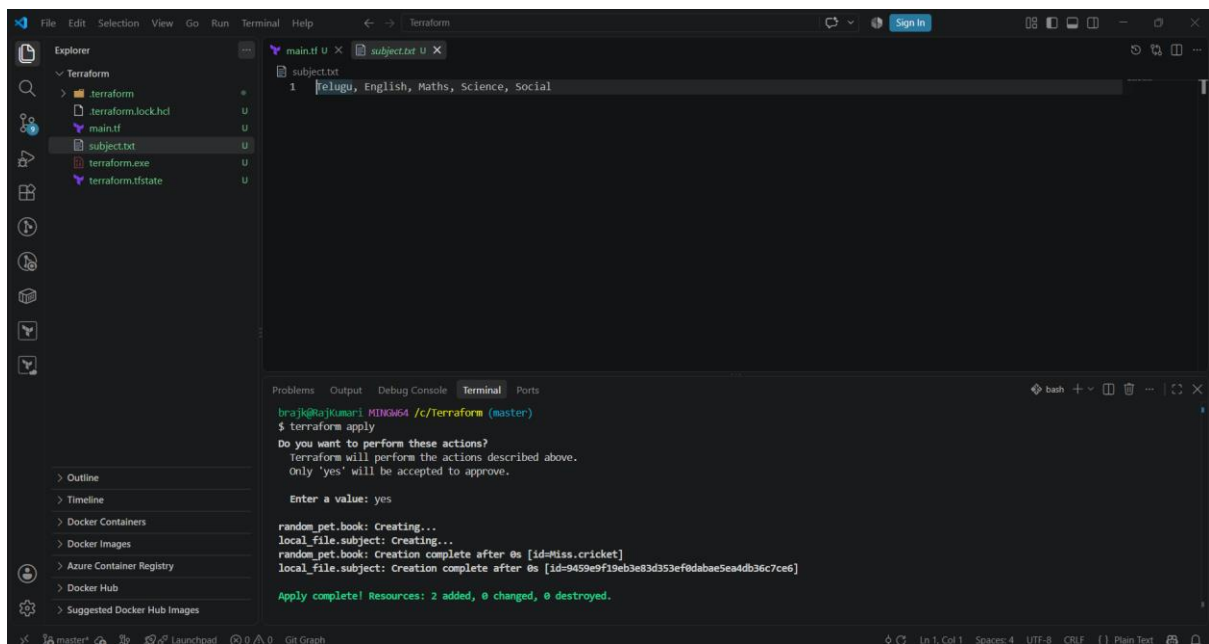
```
1 resource "local_file" "subject" {
2   filename = "subject.txt"
3   content = "Telugu, English, Maths, Science, Social"
4 }
5
6 resource "random_pet" "book" {
7   prefix = "Miss"
8   separator = "-"
9   length = 1
10 }
```

```
brajk@rajkumari-MINGW64 /c/terraform (master)
$ terraform apply
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

random_pet.book: Creating...
local_file.subject: Creating...
random_pet.book: Creation complete after 0s [id=Miss.cricket]
local_file.subject: Creation complete after 0s [id=9459e9f19eb3e83d353ef0dabae5ea4db36c7ce6]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```



This screenshot shows the VS Code interface with the subject.txt file open. The Explorer sidebar on the left shows the project structure. The main editor displays the content of subject.txt, which contains the text 'Telugu, English, Maths, Science, Social'. The terminal at the bottom shows the execution of the terraform apply command, which prompts for confirmation and then successfully creates the resources.

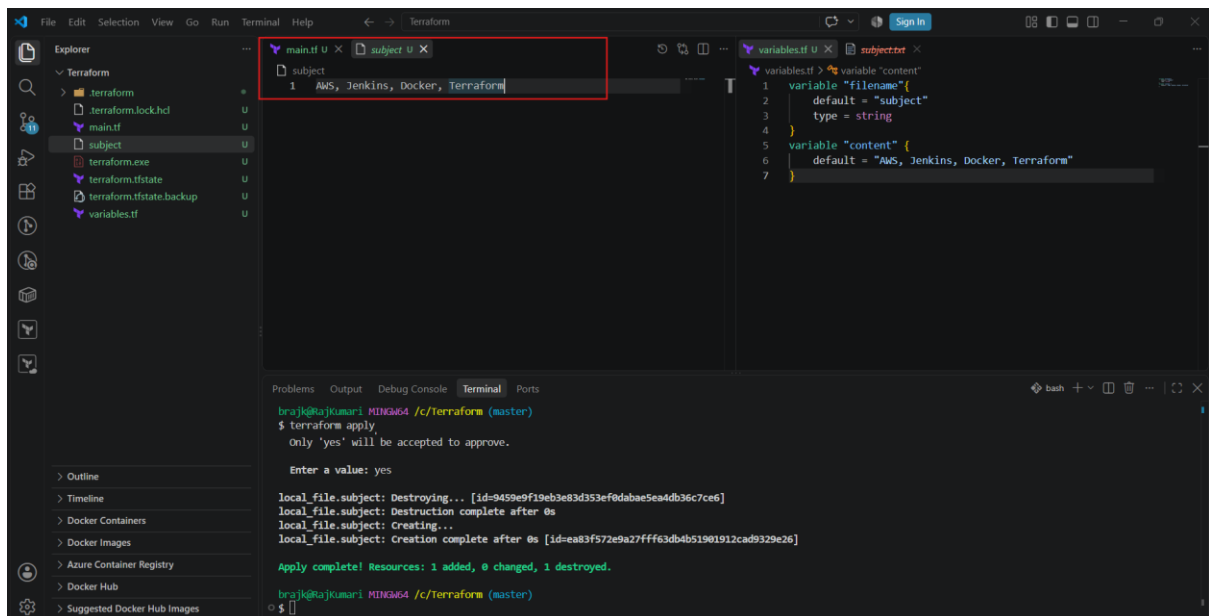
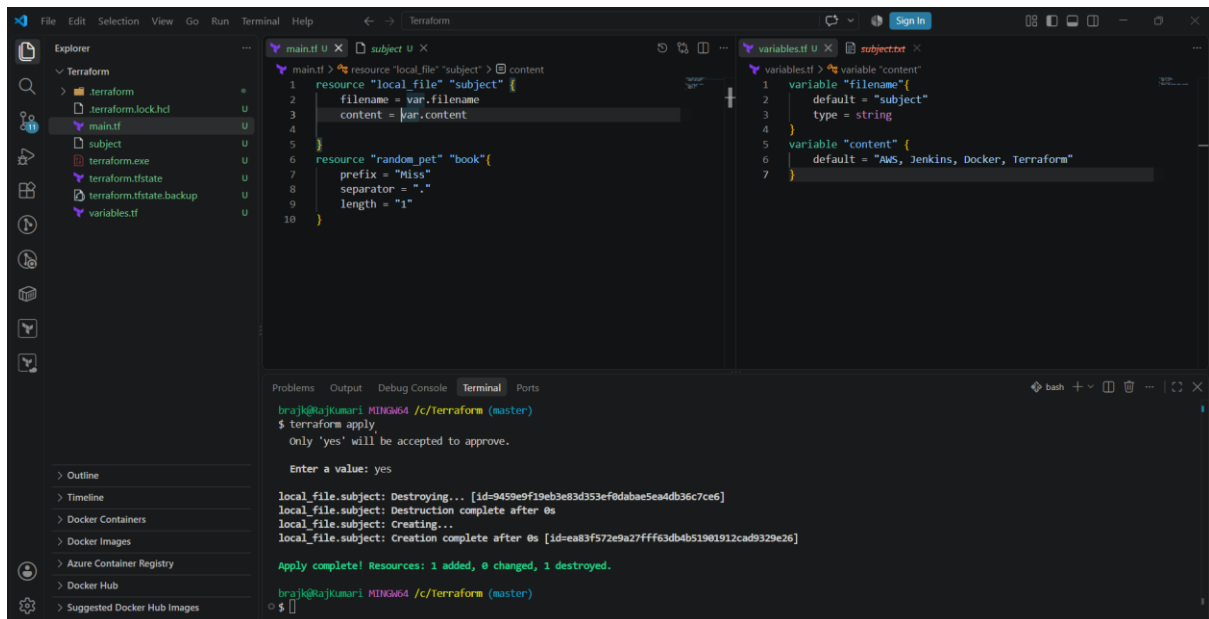
```
1 Telugu, English, Maths, Science, Social
```

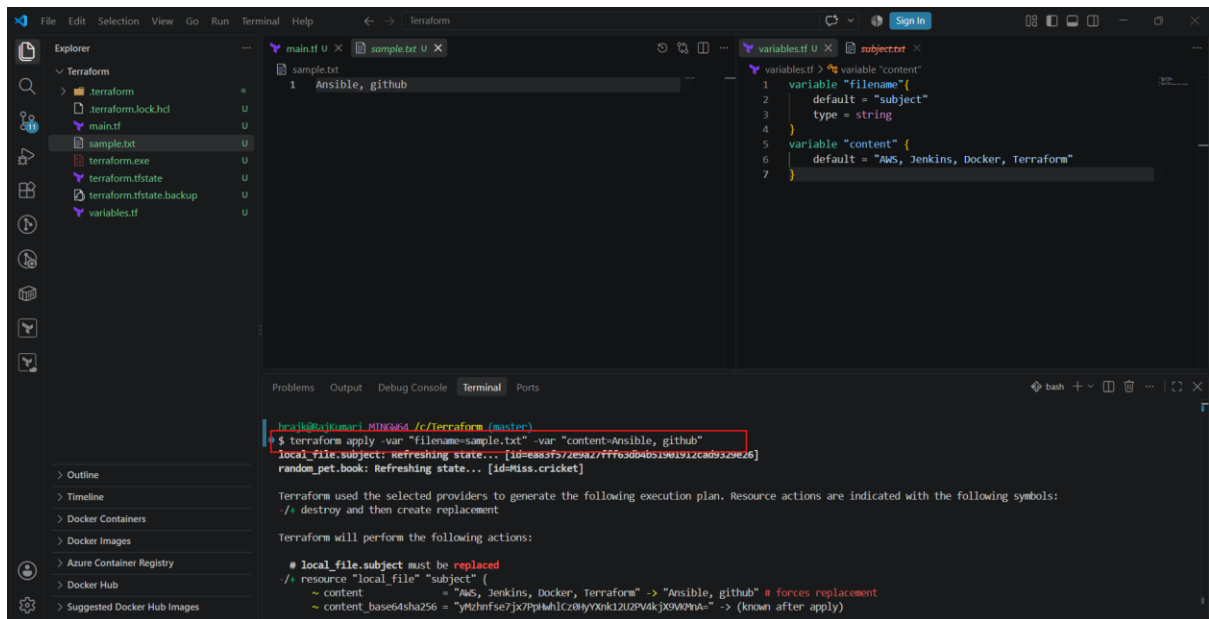
```
brajk@rajkumari-MINGW64 /c/terraform (master)
$ terraform apply
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

random_pet.book: Creating...
local_file.subject: Creating...
random_pet.book: Creation complete after 0s [id=Miss.cricket]
local_file.subject: Creation complete after 0s [id=9459e9f19eb3e83d353ef0dabae5ea4db36c7ce6]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
```



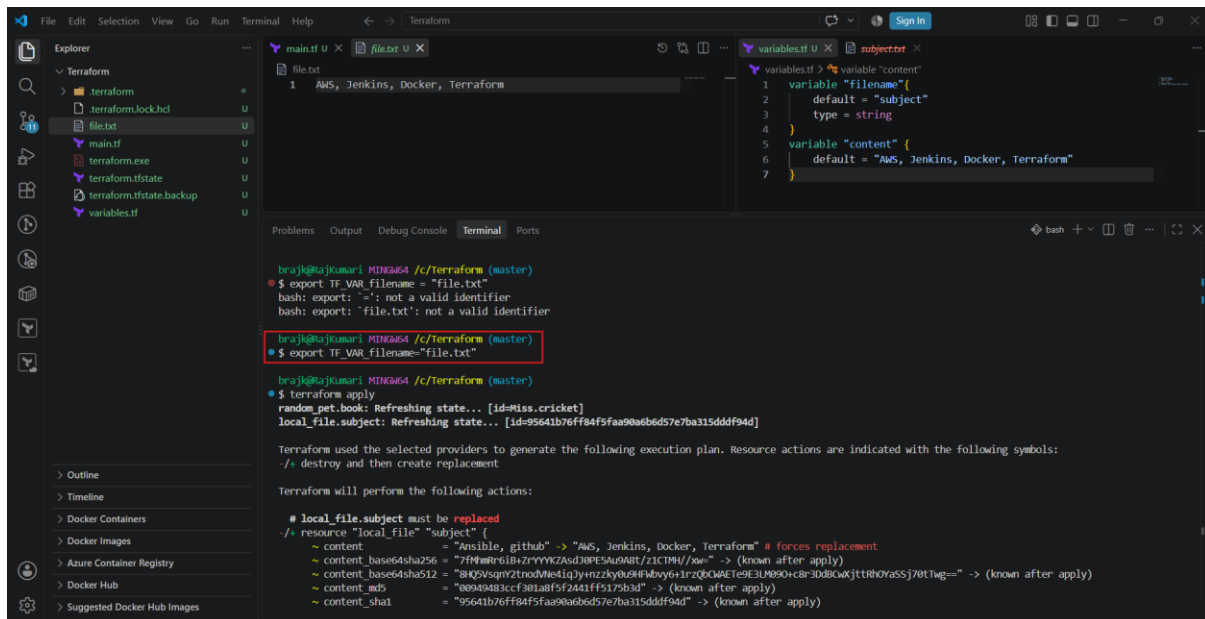


Environmental Variables in Terraform: Terraform environment variables are variables defined at the operating-system level and automatically consumed by Terraform using the **TF_VAR_ prefix to provide values** for Terraform input variables without hard-coding them in **.tf files**.

Why use them?

- Avoid hard-coding configuration values
- Different values can be used for different environments
- Useful in CI/CD pipelines such as Jenkins
- Can help keep sensitive values out of .tf files (though secrets should still be handled carefully)

Example: **export TF_VAR_filename="file.txt"**



Terraform Output Variable: A Terraform output variable is used to **display or expose information about resources created by Terraform** after terraform apply.

In simple words: Output variables allow Terraform to show important values from your infrastructure, such as an EC2 instance IP address, DNS name, VPC ID, or S3 bucket name.

Example

Suppose Terraform creates an EC2 instance:

```
resource "aws_instance" "server" {

  ami          = "ami-xxxxxxxx"

  instance_type = "t2.micro"

}
```

You can create an output variable:

```
output "instance_public_ip" {

  description = "Public IP address of the EC2 instance"

  value       = aws_instance.server.public_ip

}
```

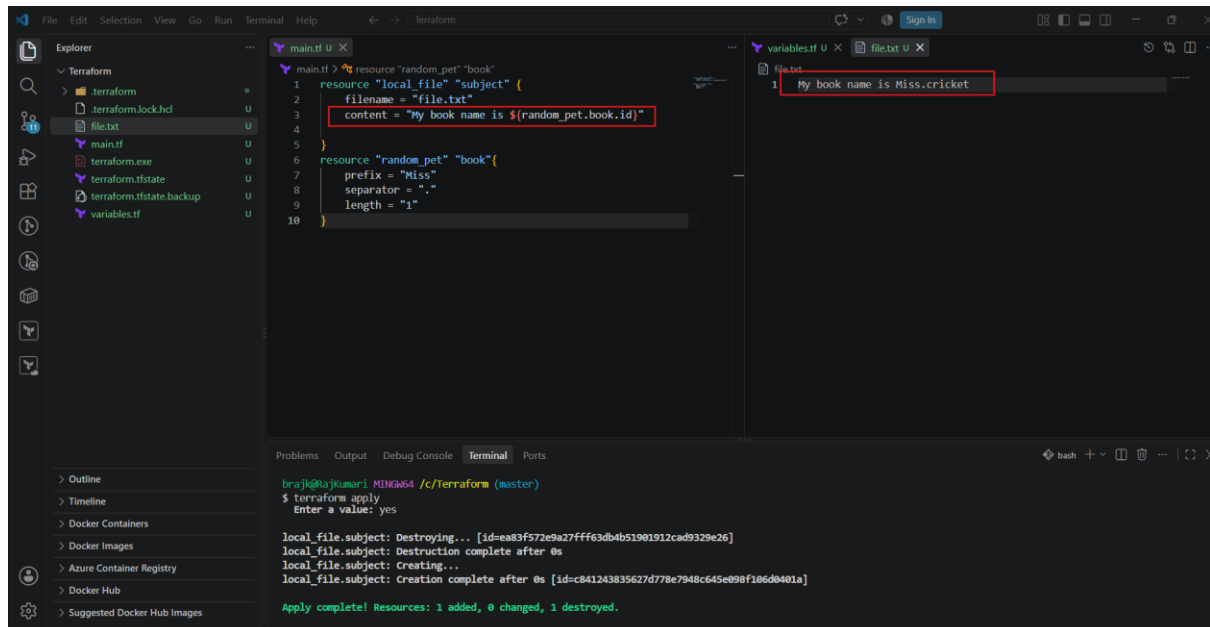
After running:

terraform apply

Terraform displays something like:

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs: instance_public_ip = "54.123.45.67"



2.Integrate Terraform in Jenkins using the Terraform plugin.

GitHub → Jenkins → Terraform Plugin → Terraform → AWS

One important point first: the Jenkins **Terraform plugin** is an older plugin (current listed version 1.0.10), and its documented configuration is primarily a **build wrapper**. For a modern Jenkins pipeline, you can still configure Terraform through **Manage Jenkins** → **Tools** and invoke Terraform from a Jenkinsfile. This is generally easier to maintain.

1. Overall architecture

Developer

|

| git push

v

GitHub Repository

|

v

Jenkins

|

| Terraform Plugin / Terraform installation

v

Terraform

|

| AWS credentials

v

AWS

|

+---- EC2

+---- VPC

+---- Security Group

+---- S3

+---- etc.

The important distinction is:

- **Jenkins Terraform plugin** → helps Jenkins integrate with Terraform.
- **Terraform AWS provider** → allows Terraform to create/manage AWS resources.

Terraform itself uses provider plugins to communicate with services such as AWS.

Step 2: Install Terraform on the Jenkins server

SSH into your Jenkins EC2 instance.

Check whether Terraform is already installed:

```
terraform version
```

If you get something like:

```
Terraform v1.x.x
```

you can proceed.

If not, install Terraform.

For example, on Amazon Linux:

```
sudo dnf update -y
```

Then install Terraform using HashiCorp's repository/package instructions appropriate for your Amazon Linux version.

After installation:

```
terraform version
```

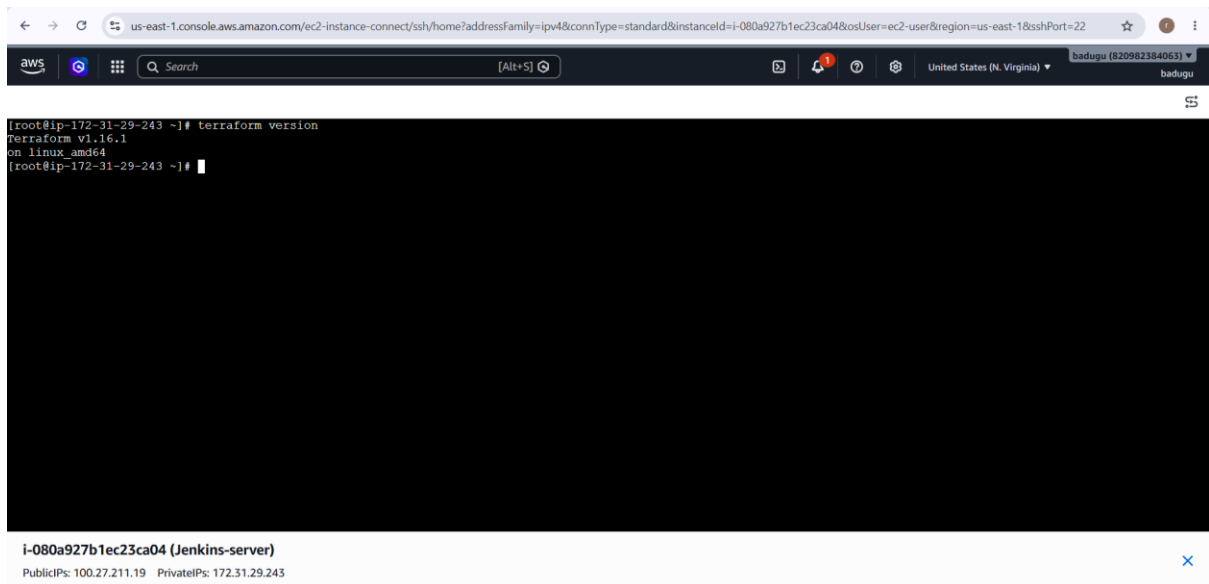
You should see:

```
Terraform v1.x.x
```

```
on linux_amd64
```

Why this is important

Jenkins ultimately needs access to the terraform executable on the machine/agent where the build runs.



Step 3: Install the Jenkins Terraform plugin

Go to:

Jenkins Dashboard



Manage Jenkins



Plugins



Available plugins

Search:

Terraform

Install:

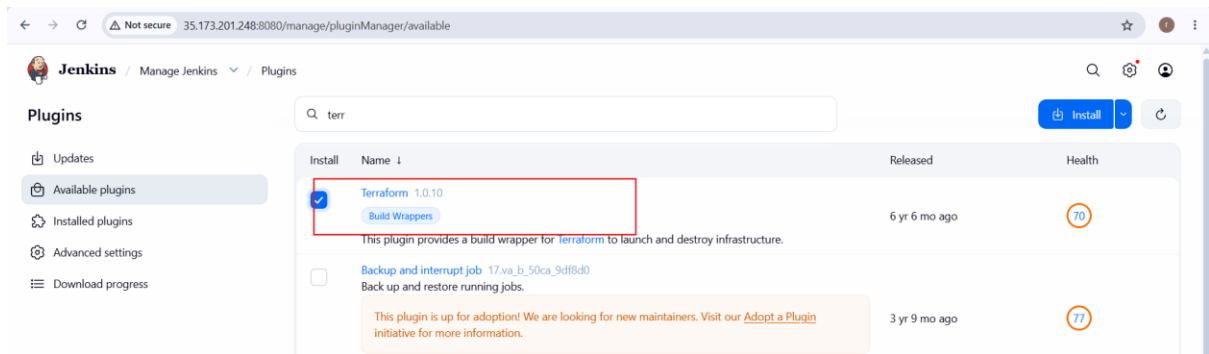
Terraform

The official Jenkins plugin page identifies the plugin ID as:

terraform

and documents installation through the Jenkins plugin manager/CLI.

After installation, restart Jenkins if Jenkins asks you to.



Step 4: Configure Terraform in Jenkins

Go to:

Jenkins Dashboard



Manage Jenkins



Tools

Look for:

Terraform installations

Click:

Add Terraform

Give it a name, for example:

terraform

or:

Terraform-1.13

You can configure the Terraform executable/installation here.

For example:

Name:

terraform

Install automatically:

✓

Or, if Terraform is already installed on your Jenkins server, configure Jenkins to use the existing Terraform installation.

Then click:

Save

The screenshot shows the Jenkins 'Manage Jenkins' configuration page for Tools. The 'Terraform installations' section is expanded, showing a configuration for 'Terraform'. The 'Name' field is 'terraform'. The 'Install automatically' checkbox is checked. Under 'Install from bintray.com', the 'Version' dropdown is set to 'Terraform 60827 linux (amd64)'. At the bottom are 'Save' and 'Apply' buttons.

3. Create one Jenkins job using Maven Project for the code below with two stages:

- **Stage 1: Git clone**

- **Stage 2: Maven Compilation Code:**

<https://github.com/betawins/java-Working-app.git>

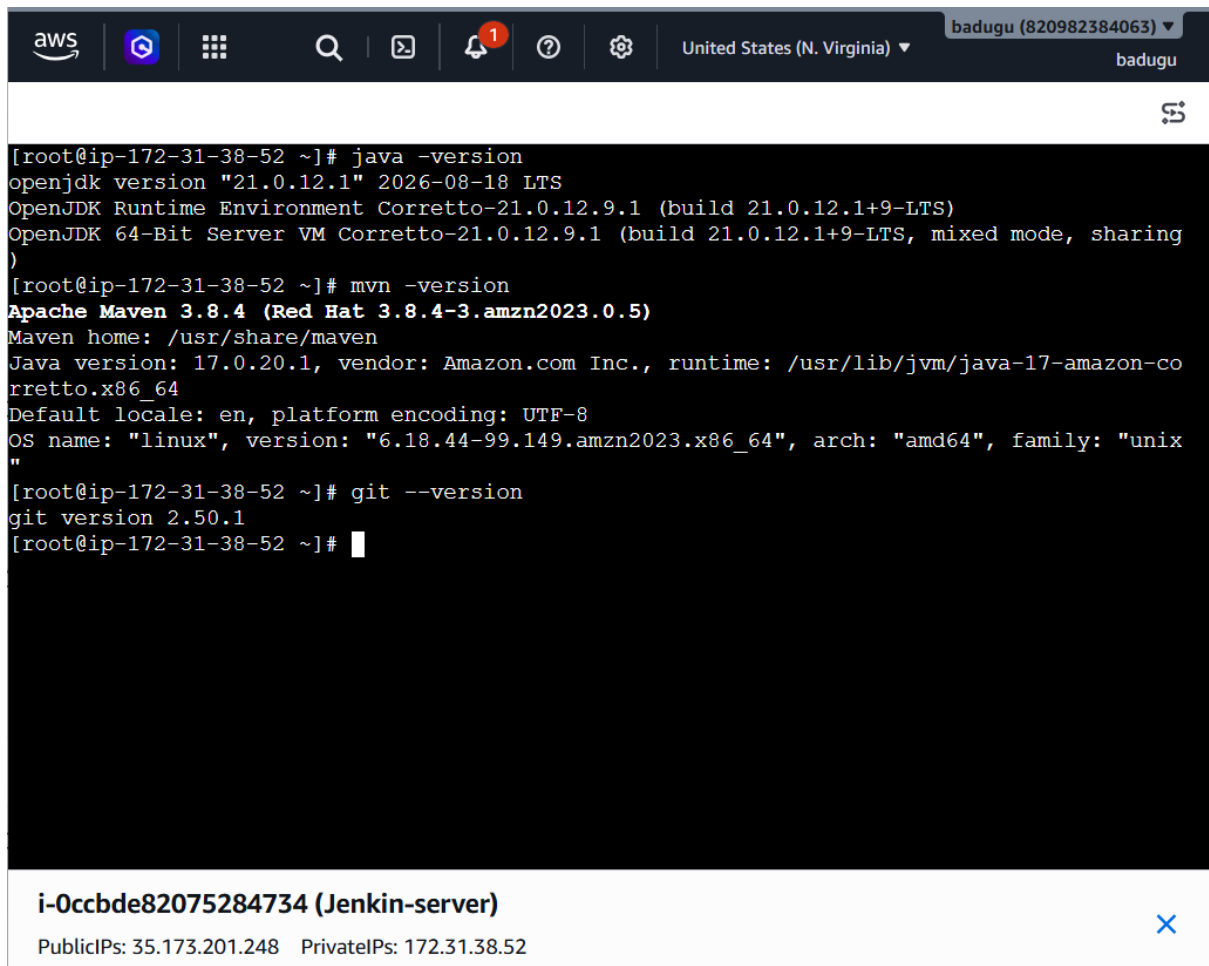
1. First verify Maven on Jenkins

SSH into your Jenkins server and run:

```
java -version
```

```
mvn -version
```

```
git --version
```



The screenshot shows an AWS CloudShell terminal window. The top bar includes the AWS logo, navigation icons, a search bar, a notification bell with a red '1', a help icon, a settings gear, and a dropdown menu for 'United States (N. Virginia)'. The user 'badugu (820982384063)' is logged in. The terminal output is as follows:

```
[root@ip-172-31-38-52 ~]# java -version
openjdk version "21.0.12.1" 2026-08-18 LTS
OpenJDK Runtime Environment Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS)
OpenJDK 64-Bit Server VM Corretto-21.0.12.9.1 (build 21.0.12.1+9-LTS, mixed mode, sharing)

[root@ip-172-31-38-52 ~]# mvn -version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.20.1, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.18.44-99.149.amzn2023.x86_64", arch: "amd64", family: "unix"

[root@ip-172-31-38-52 ~]# git --version
git version 2.50.1
[root@ip-172-31-38-52 ~]#
```

At the bottom of the terminal window, there is a status bar for the instance 'i-0ccbde82075284734 (Jenkin-server)'. It lists 'PublicIPs: 35.173.201.248' and 'PrivateIPs: 172.31.38.52'. A blue 'X' icon is visible on the right side of the status bar.

2. Configure Maven in Jenkins

Go to:

Jenkins Dashboard



Manage Jenkins



Tools

Find:

Maven installations

Click:

Add Maven

Configure:

Name: Maven-3.8.4

You can either allow Jenkins to install Maven automatically or point it to an existing Maven installation.

For example:

Maven installations

Name:

Maven-3.8.4

Version:

3.8.4

Click:

Save

The screenshot shows the Jenkins web interface at the URL `35.173.201.248:8080/manage/configureTools/`. The page title is "Jenkins / Manage Jenkins / Tools". The main content area is titled "Maven installations" and contains a list of Maven tools. A single tool named "Maven" is currently configured. It has a name field set to "Maven", an "Install automatically" checkbox checked, and an "Install from Apache" section with a version dropdown set to "3.8.4". There are buttons for "+ Add Maven" and "+ Add Installer". At the bottom, there are "Save" and "Apply" buttons.

← → ↻ ⚠ Not secure 35.173.201.248:8080/manage/configureTools/ ☆ ⓘ

Jenkins / Manage Jenkins ▾ / Tools 🔍 ⚙️ 👤

Maven installations

+ Add Maven

⋮ Maven ✕

Name

Maven

☒ Install automatically ?

⋮ Install from Apache ✕

Version

3.8.4 ▾

+ Add Installer

+ Add Maven

Save Apply

3. Configure Git

Go to:

Manage Jenkins



Tools



Git installations

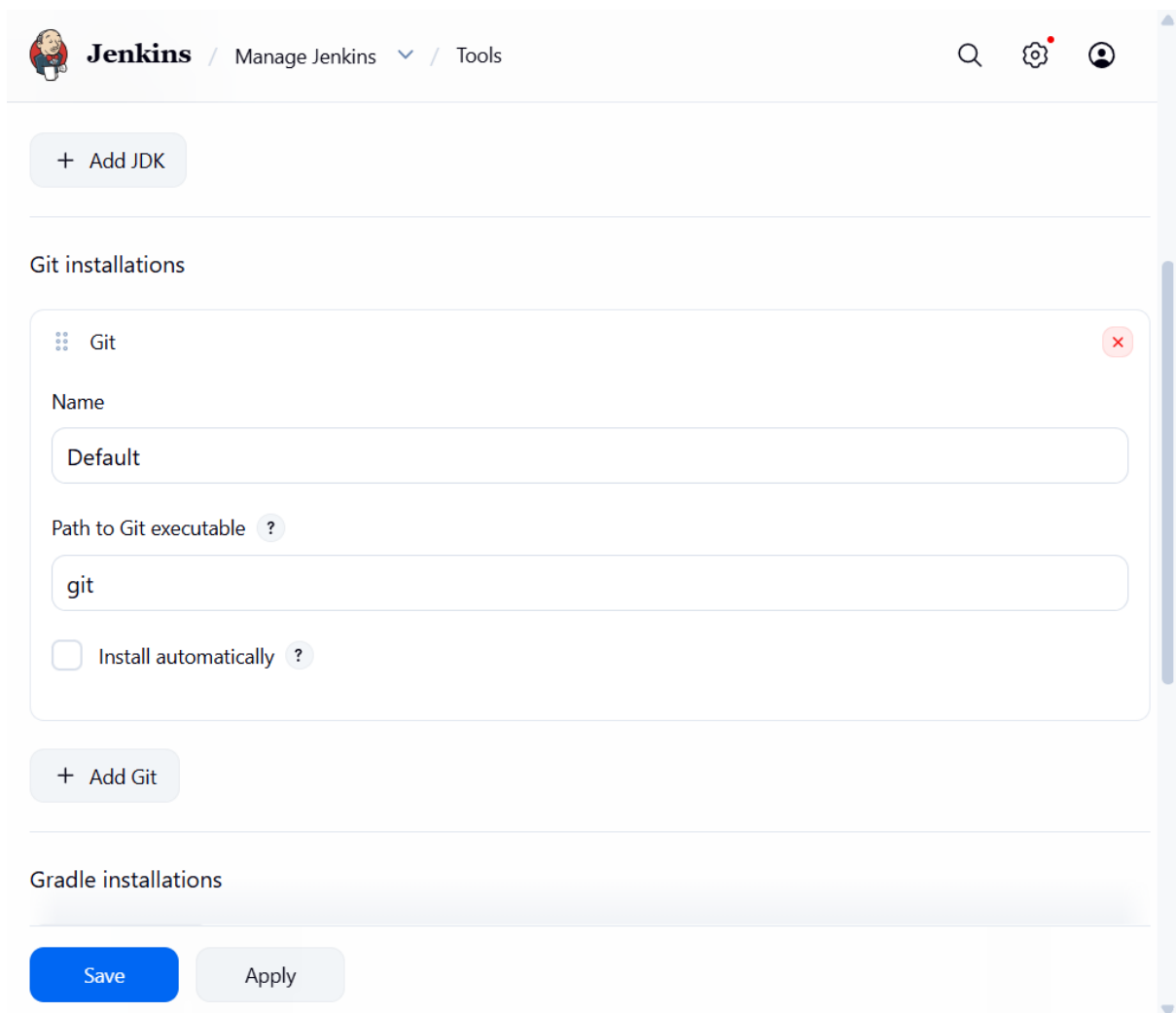
Make sure Git is configured.

For example:

Name: Default

Path to Git executable: git

If Jenkins can find Git automatically, you can leave the default configuration.



The screenshot shows the Jenkins web interface. At the top, there's a header with the Jenkins logo, the text "Jenkins", and navigation links "Manage Jenkins" and "Tools". On the right, there are icons for search, settings, and a user profile. Below the header, there's a button "+ Add JDK". The main section is titled "Git installations". It contains a single configuration entry for "Git". This entry has a "Name" field with the value "Default", a "Path to Git executable" field with the value "git", and an unchecked checkbox for "Install automatically". Below this entry is a button "+ Add Git". At the bottom of the page, there are two buttons: "Save" (in blue) and "Apply" (in light blue).

4. Create the Maven Project

Go to:

Jenkins Dashboard



New Item

Enter:

Name:

java-working-app

Select:

Maven project

Then click:

OK

You now have a **Freestyle Maven Project**.

Go to:

Manage Jenkins → Plugins → Available plugins

Search for:

Maven Integration

Install:

Maven Integration plugin

Then **restart Jenkins**.

 **Jenkins** / All / New Item

Enter an item name

java-working-app

Select an item type

 Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

 Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

 **Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

 Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

 Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the

OK

5. Stage 1 — Git Clone

In the job configuration, find:

Source Code Management

Select:

Git

Enter:

Repository URL:

<https://github.com/betawins/java-Working-app.git>

The screenshot shows the Jenkins web interface in a browser. The address bar indicates the URL is `35.173.201.248:8080/job/java-working-app/configure`. The page title is "Jenkins / java-working-app / Configure". The configuration form includes the following sections:

- Repository URL**: A text input field containing `https://github.com/betawins/java-Working-app.git`.
- Credentials**: A dropdown menu showing `- none -` and an `+ Add` button.
- Advanced**: A button with a dropdown arrow.
- + Add Repository**: A button to add a new repository.
- Branches to build**: A section with a **Branch Specifier (blank for 'any')** input field containing `*/main`.
- + Add Branch**: A button to add a new branch.

At the bottom of the form are two buttons: **Save** (in blue) and **Apply** (in light gray).

6. Stage 2 — Maven Compilation

Now scroll down to:

Build

Because you selected **Maven project**, Jenkins provides Maven build configuration.

Configure:

Root POM:

`pom.xml`

Then:

Goals and options:

`clean compile`

So your configuration should look like:

Root POM:

pom.xml

Goals and options:

clean compile

What does this do?

mvn clean compile

clean:

Deletes the previous target directory

compile:

Compiles Java source code

So:

pom.xml

↓

Maven

↓

Download dependencies

↓

Compile Java source code

↓

target/classes

7. Save the Job

Click:

Save

Then click:

Build Now

8. Expected Jenkins execution

You should see something similar to:

Started by user Rajkumari

Building in workspace:

/var/lib/jenkins/workspace/java-working-app

[Git] Cloning repository

<https://github.com/betawins/java-Working-app.git>

Checking out Revision xxxxx

[INFO] Scanning for projects...

[INFO] --- maven-clean-plugin ---

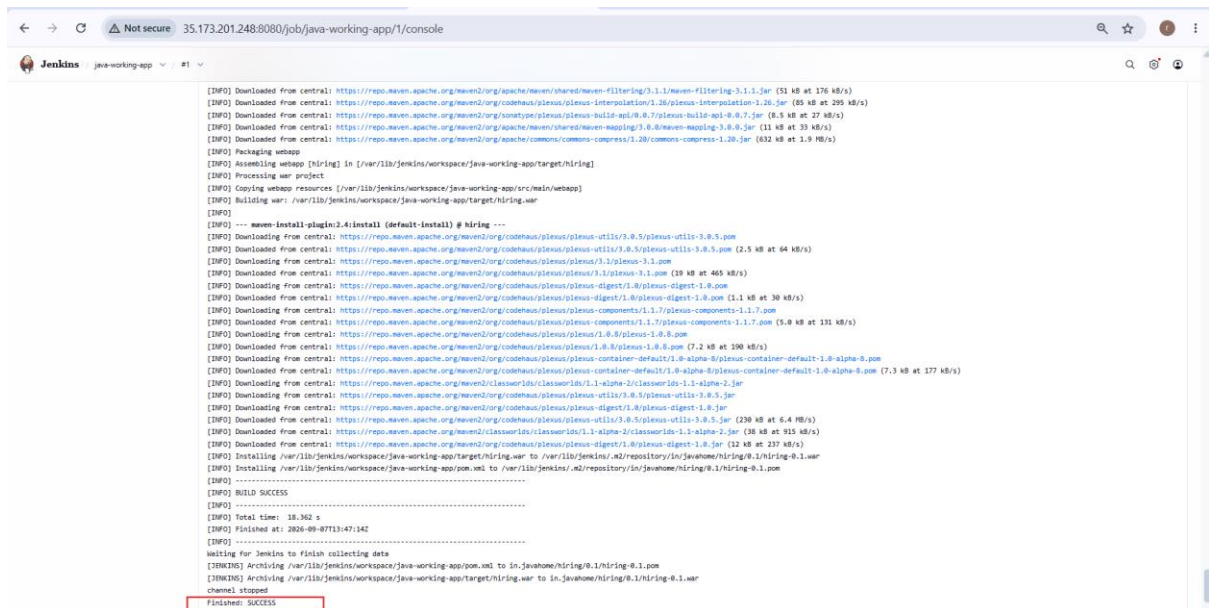
[INFO] Deleting target

[INFO] --- maven-compiler-plugin ---

[INFO] Compiling Java sources

[INFO] BUILD SUCCESS

Finished: SUCCESS



```
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-filtering/3.1.1/maven-filtering-3.1.1.jar (51 kB at 176 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-interpolation/1.16/plexus-interpolation-1.16.jar (85 kB at 295 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-build-api/0.0.7/plexus-build-api-0.0.7.jar (8.5 kB at 27 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-mapping/3.0.0/maven-mapping-3.0.0.jar (11 kB at 33 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/commons/commons-compress/1.20/commons-compress-1.20.jar (632 kB at 1.9 MB/s)
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/java-working-app/target/hiring]
[INFO] Processing project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/java-working-app/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/java-working-app/target/hiring.war
[INFO]
[INFO] --- maven-install-plugin:2.4:install (default-install) @ hiring ---
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.pom
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.pom (2.5 kB at 64 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/3.1/plexus-3.1.pom
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/3.1/plexus-3.1.pom (19 kB at 465 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.pom
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.pom (1.1 kB at 30 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-components/1.1.7/plexus-components-1.1.7.pom
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-components/1.1.7/plexus-components-1.1.7.pom (5.0 kB at 131 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/1.0.8/plexus-1.0.8.pom
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus/1.0.8/plexus-1.0.8.pom (7.2 kB at 190 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-container-default/1.0-alpha-8/plexus-container-default-1.0-alpha-8.pom
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-container-default/1.0-alpha-8/plexus-container-default-1.0-alpha-8.pom (7.3 kB at 177 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/classworlds/classworlds/1.1-alpha-2/classworlds-1.1-alpha-2.jar
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.jar
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.jar
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.jar (12 kB at 237 kB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.jar (230 kB at 6.4 MB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/classworlds/classworlds/1.1-alpha-2/classworlds-1.1-alpha-2.jar (38 kB at 915 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-digest/1.0/plexus-digest-1.0.jar (12 kB at 237 kB/s)
[INFO] Installing /var/lib/jenkins/workspace/java-working-app/target/hiring.war to /var/lib/jenkins/m2/repository/in/javahome/hiring/0.1/hiring-0.1.war
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 18.362 s
[INFO] Finished at: 2026-09-07T13:47:14Z
[INFO] -----
[INFO] Waiting for Jenkins to finish collecting data
[INFO] Archiving /var/lib/jenkins/workspace/java-working-app/pom.xml to in.javahome/hiring/0.1/hiring-0.1.pom
[INFO] Archiving /var/lib/jenkins/workspace/java-working-app/target/hiring.war to in.javahome/hiring/0.1/hiring-0.1.war
channel stopped
Finisher: SUCCESS
```

4. Use the same code and create a parameterized job in Jenkins with:

- Stage 1: Git clone
- Stage 2: Maven Compilation

<https://github.com/betawins/java-Working-app.git>

1. Create the job

Go to:

Jenkins Dashboard



New Item

Enter:

Name:

java-working-app-parameterized

Select:

Maven project

Click **OK**.

← → ↻ ⚠ Not secure 35.173.201.248:8080/view/all/newJob 🔍 ☆ 👤 ⋮

Jenkins / All ▾ / New Item 🔍 ⚙️ 👤

New Item

Enter an item name

java-working-app-parameterized

Select an item type

- Pipeline**
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

OK

2. Enable parameterization

In the job configuration, find:

General

Select:

☒ This project is parameterized

Now click:

Add Parameter

We will create a **String Parameter** for the Git branch.

← → ↻ ⚠ Not secure 35.173.201.248:8080/job/java-working-app-parameterized/configure 🔍 ☆ 🧑

Jenkins / java-working-app-parame... / Configure 🔍 ⚙️ 🧑

Configure

- ⚙️ General
- 🔑 Source Code Management
- 🕒 Triggers
- 🌐 Environment
- ⚙️ Pre Steps
- ⚙️ Build
- ⚙️ Post Steps
- ⚙️ Build Settings
- 📦 Post-build Actions

Plain text [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☒ This project is parameterized ?

⋮ String Parameter ?

Name ?

BRANCH

Default Value ?

main

Description ?

Git branch to clone

Plain text [Preview](#)

☐ Trim the string ?

[Save](#) [Apply](#)

4. Create MAVEN_GOAL parameter

Again:

Add Parameter



String Parameter

Configure:

Name:

MAVEN_GOAL

Default Value:

clean compile

Description:

Maven goals to execute

Now you have:

BRANCH

main

MAVEN_GOAL

clean compile

← → ↻ ⚠ Not secure 35.173.201.248:8080/job/java-working-app-parameterized/configure 🔍 ☆ 👤

Jenkins / java-working-app-parame... / Configure 🔍 ⚙️ 👤

Configure

- ⚙️ General
- 🔑 Source Code Management
- 🕒 Triggers
- 🌐 Environment
- ⚙️ Pre Steps
- ⚙️ Build
- ⚙️ Post Steps
- ⚙️ Build Settings
- 📦 Post-build Actions

Plain text [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☒ This project is parameterized ?

⚙️ String Parameter ?

Name ?

BRANCH

Default Value ?

main

Description ?

Git branch to clone

Plain text [Preview](#)

☐ Trim the string ?

[Save](#) [Apply](#)

5. Configure Stage 1 — Git Clone

Go to:

Source Code Management

Select:

Git

Repository URL:

<https://github.com/betawins/java-Working-app.git>

For **Branch Specifier**, instead of hardcoding:

*/main

use:

*\${BRANCH}

The screenshot shows the Jenkins configuration interface for a job named 'java-working-app-parameterized'. The left sidebar lists various configuration sections: General, Source Code Management (selected), Triggers, Environment, Pre Steps, Build, Post Steps, Build Settings, and Post-build Actions. The main content area is titled 'Configure' and includes a 'for your builds.' section with radio buttons for 'None' and 'Git' (selected). Below this, the 'Repositories' section contains a 'Repository URL' field with the value 'https://github.com/betawins/java-Working-app.git', a 'Credentials' dropdown menu set to '- none -', and an 'Advanced' dropdown. An 'Add Repository' button is located below the repository settings. The 'Branches to build' section features a 'Branch Specifier (blank for \'any\')' field with the value '*/main'. At the bottom, there are 'Save' and 'Apply' buttons.

6. Configure Stage 2 — Maven Compilation

Go to:

Build

Because this is a **Maven project**, configure:

Root POM

pom.xml

Goals and options

Use:

`${MAVEN_GOAL}`

So:

Root POM:

`pom.xml`

Goals and options:

`${MAVEN_GOAL}`

When you select the default:

`MAVEN_GOAL = clean compile`

Jenkins executes:

`mvn clean compile`

6. Configure Stage 2 — Maven Compilation

Go to:

Build

Because this is a **Maven project**, configure:

Root POM

`pom.xml`

Goals and options

Use:

`${MAVEN_GOAL}`

So:

Root POM:

`pom.xml`

Goals and options:

`${MAVEN_GOAL}`

When you select the default:

`MAVEN_GOAL = clean compile`

Jenkins executes:

`mvn clean compile`

7. Configure Maven installation

Make sure Jenkins has Maven configured.

Go to:

Manage Jenkins



Tools



Maven installations

For example:

Name:

Maven-3.8.4

Then in your Maven project configuration select:

Maven Version:

Maven-3.8.4

8. Save

Click:

Save

Now instead of **Build Now**, you should see:

Build with Parameters

Click it.

9. Run with parameters

You should see:

Build with Parameters

BRANCH

main

MAVEN_GOAL

clean compile

[Build]

Leave the defaults:

BRANCH = main

MAVEN_GOAL = clean compile

Click:

Build

10. Jenkins execution

Stage 1 — Git Clone

Jenkins effectively performs:

Repository:

<https://github.com/betawins/java-Working-app.git>

Branch:

main

and checks the code out into:

/var/lib/jenkins/workspace/java-working-app-parameterized

Stage 2 — Maven Compilation

Jenkins reads:

pom.xml

and executes:

mvn clean compile

The flow is:

pom.xml

↓

Maven

↓

clean

↓

compile

↓

Java source files

↓

target/classes

If everything is successful:

BUILD SUCCESS

11. Try changing the parameter

Now click:

Build with Parameters

You can change:

BRANCH = development

and:

MAVEN_GOAL = clean package

Then Jenkins will execute approximately:

Git Clone



development branch



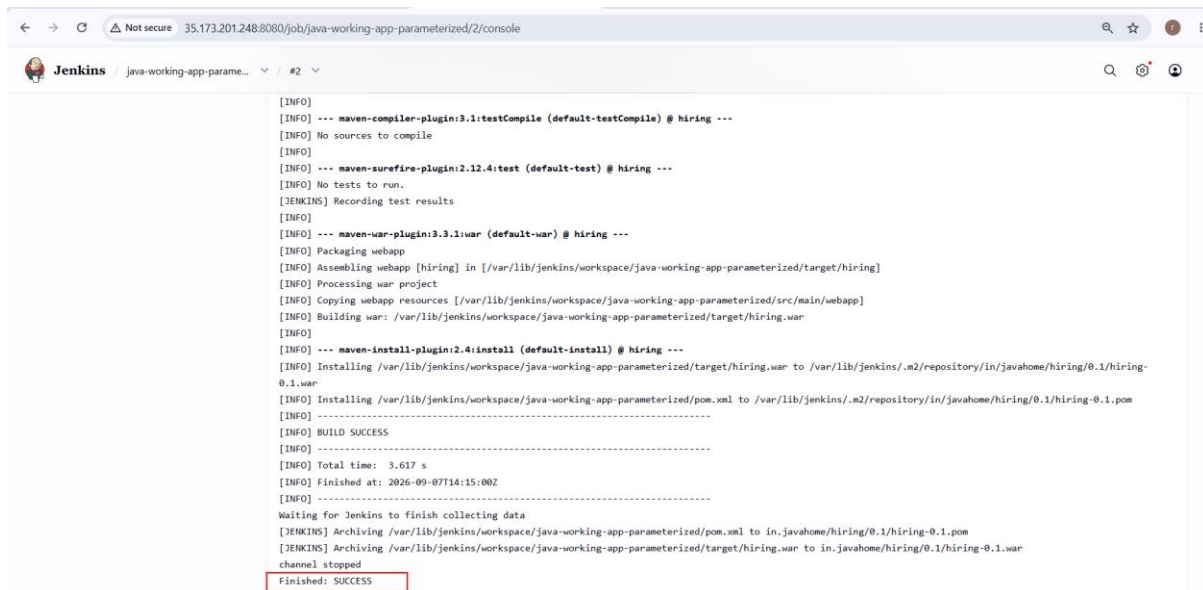
Maven



mvn clean package

So the same Jenkins job can be reused for different branches and Maven operations.

The screenshot shows the Jenkins web interface for a job named 'java-working-app-parameterized'. The browser address bar indicates the URL is '35.173.201.248:8080/job/java-working-app-parameterized/'. The Jenkins logo and job name are at the top. A 'Status' tab is selected, showing a green checkmark icon and the job name. Below the status, there is a 'Permalinks' section with four links: 'Last build (#2), 46 sec ago', 'Last stable build (#2), 46 sec ago', 'Last successful build (#2), 46 sec ago', and 'Last completed build (#2), 46 sec ago'. On the left sidebar, there are links for 'Changes', 'Workspace', 'Build with Parameters', 'Delete Maven project', 'Modules', 'Configure', and 'Rename'. At the bottom, a 'Builds' section shows a list of builds for 'Today': build #2 at 2:14 PM and build #1 at 2:13 PM, both with green checkmark icons.



```
[INFO] --- maven-compiler-plugin:3.1:testCompile (default-testCompile) @ hiring ---
[INFO] No sources to compile
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ hiring ---
[INFO] No tests to run.
[INFO] Recording test results
[INFO] --- maven-war-plugin:3.3.1:war (default-war) @ hiring ---
[INFO] Packaging webapp
[INFO] Assembling webapp [hiring] in [/var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring]
[INFO] Processing war project
[INFO] Copying webapp resources [/var/lib/jenkins/workspace/java-working-app-parameterized/src/main/webapp]
[INFO] Building war: /var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring.war
[INFO] --- maven-install-plugin:2.4:install (default-install) @ hiring ---
[INFO] Installing /var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring.war to /var/lib/jenkins/.m2/repository/in/javahome/hiring/0.1/hiring-0.1.war
[INFO] Installing /var/lib/jenkins/workspace/java-working-app-parameterized/pom.xml to /var/lib/jenkins/.m2/repository/in/javahome/hiring/0.1/hiring-0.1.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.617 s
[INFO] Finished at: 2026-09-07T14:15:00Z
[INFO] -----
Waiting for Jenkins to finish collecting data
[INFO] Archiving /var/lib/jenkins/workspace/java-working-app-parameterized/pom.xml to in.javahome/hiring/0.1/hiring-0.1.pom
[INFO] Archiving /var/lib/jenkins/workspace/java-working-app-parameterized/target/hiring.war to in.javahome/hiring/0.1/hiring-0.1.war
channel stopped
Finished: SUCCESS
```

5.What are the global variables in Jenkins?

In Jenkins, **global variables** are variables that are available across Jenkins jobs or throughout a Pipeline, depending on how they are defined.

There are two common meanings of "global variables" in Jenkins:

1. Built-in Jenkins environment variables

Jenkins automatically provides many environment variables.

Examples:

Variable	Meaning	Example
BUILD_NUMBER	Current	25
	build number	
BUILD_ID	ID of the current build	25
BUILD_URL	URL of the build	http://jenkins:8080/job/demo/25/

Variable	Meaning	Example
JOB_NAME	Name of the Jenkins job	java-working-app
WORKSPACE	Workspace directory	/var/lib/jenkins/workspace/java-working-app
NODE_NAME	Jenkins node/agent name	built-in
JENKINS_HOME	Jenkins home directory	/var/lib/jenkins
GIT_COMMIT	Git commit checked out	a82f91...
GIT_BRANCH	Git branch	origin/main

You can see them in a Pipeline using:

```

pipeline {
    agent any

    stages {
        stage('Environment') {
            steps {
                sh 'echo "Job: $JOB_NAME"'
                sh 'echo "Build: $BUILD_NUMBER"'
                sh 'echo "Workspace: $WORKSPACE"'
            }
        }
    }
}

```



```
}  
}
```

2. User-defined global variables

You can also define variables that you want Jenkins to make available broadly.

For example:

JAVA_HOME

MAVEN_HOME

TOMCAT_HOME

These can be configured through Jenkins or the operating-system environment.

For example:

MAVEN_HOME=/opt/maven

JAVA_HOME=/usr/lib/jvm/java-17

Then a Jenkins job can use them.

Environment variables vs global variables

A useful way to remember this:

Jenkins

```
|  
  
+-- Built-in environment variables  
  
|   |  
  
|   +-- BUILD_NUMBER  
  
|   +-- JOB_NAME  
  
|   +-- WORKSPACE  
  
|   +-- BUILD_URL
```

```
|  
  
+-- User-defined variables  
  
|  
  
+-- JAVA_HOME  
  
+-- MAVEN_HOME  
  
+-- TOMCAT_HOME
```

In your Maven job

For your java-Working-app project, you could use:

JOB_NAME

BUILD_NUMBER

WORKSPACE

GIT_BRANCH

GIT_COMMIT

For example:

```
echo "Job Name: $JOB_NAME"
```

```
echo "Build Number: $BUILD_NUMBER"
```

```
echo "Workspace: $WORKSPACE"
```

```
echo "Git Branch: $GIT_BRANCH"
```

Global variables in Jenkins are variables that provide information or configuration values that can be accessed by Jenkins jobs or Pipeline executions. Jenkins provides built-in environment variables such as BUILD_NUMBER, JOB_NAME, and WORKSPACE, and administrators can also define custom environment variables for use across jobs.

